

高程

太好了,你终于来了
你全身都报错了,没事吧?

你没有来北楼上课

作业也都没有交

我今天是来告诉大家一件事的

我要退出汉诺塔

为什么?发生什么事了吗

汉诺塔の战斗体验报告

班级: 计拔

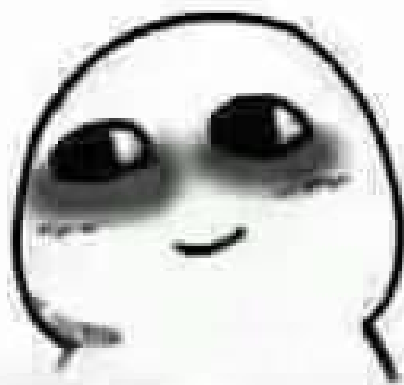
学号: 2351520

姓名: 毛星博

战斗开始时: 2024年5月11日晚

战斗胜利时: 2024年5月13日晚

崩溃次数: 记不清了



微笑中透露着同心同德同
舟楫济人济世济天下

1. 题目：汉诺塔综合演示（共9种模式选择）

1.1. 菜单实现模式显示和选择

1.1.1. 在运行程序时清屏并显示菜单1-9选项和0退出选项，用户输入对应数字选择模式或输入0选择退出。

1.2. 功能实现9种模式的要求

1.2.1. 基本解：输入层数、起始柱和目标柱，输出盘号和移动方式。

1.2.2. 基本解（步数记录）：在菜单1的基础上每行开头加上步数统计。

1.2.3. 内部数组显示（横向）：在菜单2基础上每行显示三个柱子上盘子的编号。

1.2.4. 内部数组显示（纵向+横向）：输入层数、起始柱和目标柱，选择手动按回车或5档自动移动速度，在菜单3的基础上用纵向打印三个柱子和其上的盘号显示移动过程。

1.2.5. 图形解-预备-画三个圆柱：清屏，用色块画出三个圆柱（需要加延时）。

1.2.6. 图形解-预备-在起始柱上画n个盘子：输入层数、起始柱和目标柱，在菜单5的基础上在起始柱上用不同色块画出n个盘子（需要加延时）。

1.2.7. 图形解-预备-第一次移动：在菜单6的基础上实现第一步移动过程（要求有盘子上移-平移-下移的过程）。

1.2.8. 图形解-自动移动版本：输入层数、起始柱和目标柱，选择手动按回车或5档自动移动速度，清屏，同步显示每步的图形移动过程、纵向数组变化以及菜单3的功能。

1.2.9. 图形解-游戏版：输入层数、起始柱和目标柱后，由用户手动输入每步的移动方向，同步显示每步的图形移动过程、纵向数组变化以及菜单3的功能，直到用户完成移动目标或者手动选择退出，结束程序。

1.3. 限制要求

1.3.1. 整个程序只允许使用一个递归函数，即菜单项 1/2/3/4/8 必须共用一个递归函数，递归函数要求行数不超过 15 行。

- 1.3.2. 菜单项 1/2/3/4/6/7/8 中的输入多个参数必须共用一个函数
- 1.3.3. 菜单项 3/4/8 中的横向输出必须共用一个函数，用参数解决输出位置等差异
- 1.3.4. 菜单项 4/8 中的纵向输出必须共用一个函数，用参数解决输出位置等差异
- 1.3.5. 菜单项 5/6/7/8/9 中画三个柱子的必须共用一个函数
- 1.3.6. 菜单项 7/8/9 中盘子的移动必须共用一个函数

2. 整体设计思路

首先，菜单功能显示和实现集成在一个菜单函数中，在该函数内显示菜单选项和输入以及输入错误处理，该函数返回值即为菜单模式。

设置静态全局变量：用step表示移动步数，用sum[3]表示三个圆柱的圆盘数量，用num[3][10]表示三个圆柱的圆盘编号，用speed表示延时。

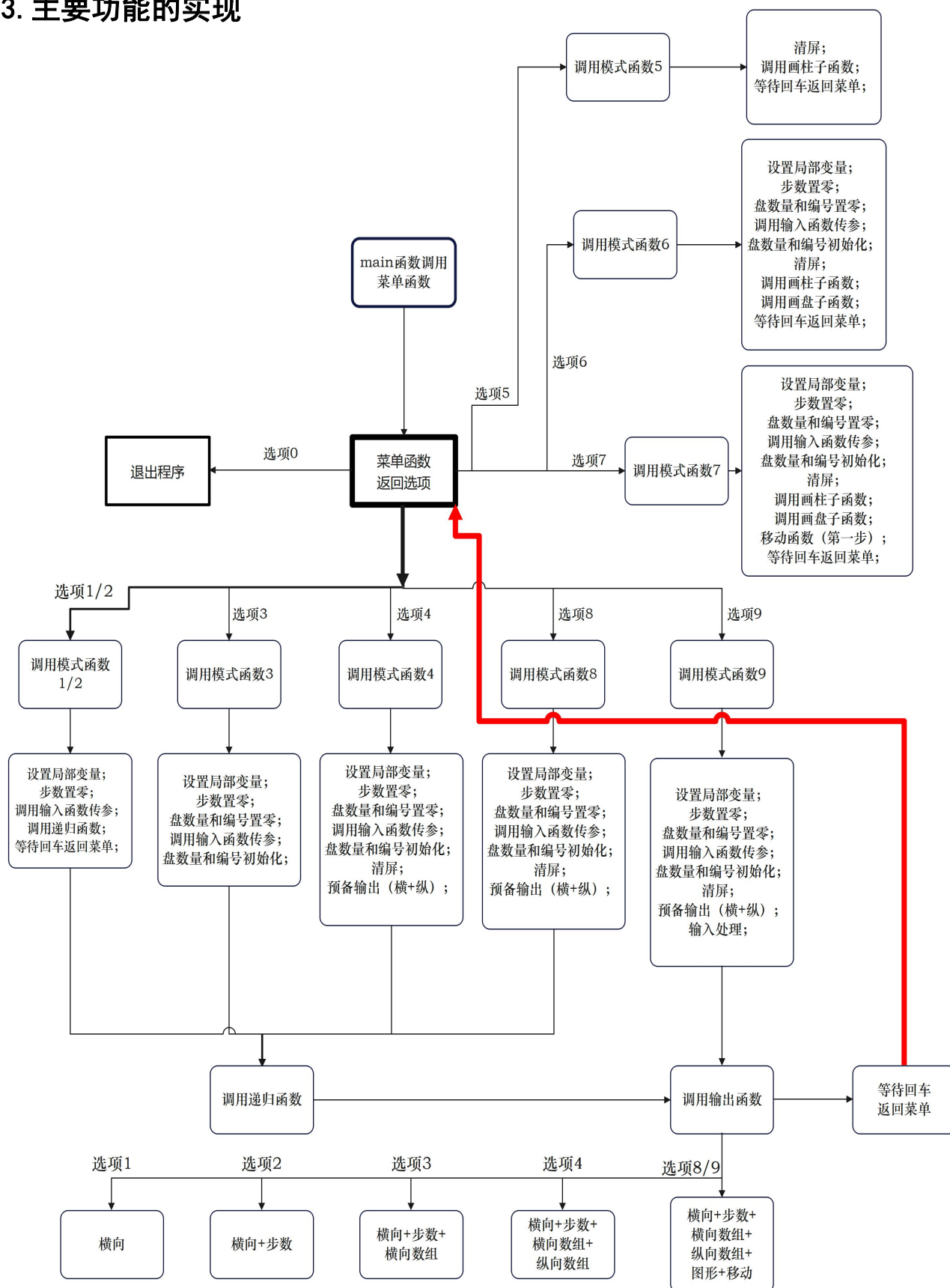
然后进行最主要的函数：汉诺塔递归函数的编写。由于该函数要求行数不超过15行，因此只保留最基本的递归语句，其余均通过在该递归函数中调用输出函数实现，不同模式的不同输出方式通过一个flag参数控制。由于要求菜单项 3/4/8 中的横向输出必须共用一个函数，菜单项 4/8 中的纵向输出必须共用一个函数，因此我决定直接将所有菜单项的输出放在一个输出函数中（递归函数中调用的即该函数），通过参数决定各菜单项的横向输出、纵向输出和画盘子、柱子和移动过程的实现。在编写输出函数的过程中逐步完善其他函数，比如速度处理、画柱子和盘子的函数、盘子移动的函数等等。

然后进行输入函数的编写。由于要求菜单的输入必须用同一个函数，且该函数需要有多个数据记录，因此需要使用指针（函数形参为实参的指针），以此同时改变多个实参值。输入函数中需要实现汉诺塔层数、起始柱和目标柱，以及可能需要的移动速度的输入（是否输入用参数决定），同时需要处理各个输入的错误输入处理。最后通过指针将各数据传回调用该函数时填入的实参中。

然后，需要编写 9 个函数，对应实现 9 个菜单项的功能。在每个模式函数中定义所需局部变量并调用输入函数获取数据。同时需要注意在每个模式函数的开头要将全局变量重置，然后进行初始设置，比如模式 3/4/8/9 中需要在调用递归函数前设置全局变量的初始圆盘数量和编号，以便于多次使用。然后再进行递归等功能（如果需要）。实现完函数后，需要设置一个函数来等待输入回车后继续返回菜单。

最后，在 main 文件中先调用菜单函数获取菜单项，再调用对应的模式函数即可实现各个菜单项功能。

3. 主要功能的实现



4. 调试过程碰到的问题

问题1：写盘子移动函数时如何确定光标移动的起始位置和终止位置。

解决方法：首先将延时时间调长，运行函数并观察盘子移动的位置，确定问题出现的代码块位置，然后通过草稿纸上模拟某个盘子的移动，确定光标的上移、平移和下移的起始位置和终止位置与起始柱、目标柱和其上第一个盘子的大小的关系，逐步确定循环的起止条件，再通过程序试错，逐步修改bug和各项条件，最终定光标移动的起始位置和终止位置。

问题2：在模式9的自主移动输入中，如何实现demo的输入有回显，输入超过一定数量的字符会自动清屏重新输入，且输入数量或字符不符合要求时清屏并重新输入。

解决方法：首先考虑条件：需要读取输入时有回显，且能够读取回车（即换行符\n），查询资料得知，函数_getche满足以上条件（注意使用该函数需要包含头文件<conio.h>）。然后考虑输入超过一定数量的字符会自动清屏重新输入，因此采用定义字符数组并循环输入的方式获取输入，通过换行符在字符数组的位置判断输入错误以及退出游戏的条件。

问题3：程序要求多个菜单项使用同一个递归函数，同一个输出函数，但每个菜单项的功能有差异，如何解决？

解决方法：通过对同一个函数传入参数实现。不同的菜单项调用函数时传入不同参数，函数根据不同的参数实现不同的功能。

5. 心得体会

5.1 完成本次作业得到的一些心得体会，经验教训

首先，当我们拿到一个代码的题目时，不要急于开始写代码，而应该先全面理解问题的需求，明确需要解决的问题是什么，问题的输入和输出是什么，以及有哪些限制条件。最好可以在草稿纸上先初步分析解决流程和标注限制条件，并初步将大型代码分成多个实现目标，再通过函数逐个实现功能，最后再整合。

其次，在函数中使用一些重复度高的数据时，可以用一个变量替代它，这样不仅使代码更精简，同时在发现该数据需要修改时，可以方便快捷的改掉所有需要使用的地方，减少维护的时间成本。

在对一个变量或者一个函数命名时，命名要与变量或者函数的功能匹配，这样在后续的修改维护中才能使代码有更好的可读性，不至于需要重新分析函数的功能。

要养成写注释的好习惯！

在测试代码时，不一定要按照最终目标测试，要适当使用输出中间变量来进行更直观的测试。同时，测试不能太过简单，要考虑可能出现的所有情况，尤其是边界条件和一些可能的异常值都要测试一遍，避免漏掉一些 bug。

在编写代码过程中，会遇到很多 bug，有时候会很让人崩溃。这时候切记要沉下心来分析和调试代码，分析问题出在哪里。大部分时候，代码的大框架不会有大问题，程序有问题很可能只是其中一小块甚至一个数值的问题。可以选取某个变量，在草稿纸上模拟代码实现过程，逐步修正代码。保持良好心态是解决问题的关键！！

5.2 在做一些较为复杂的程序时，是分为若干小题还是直接一道大题的形式更好？

我认为将复杂问题分解为若干小题更为合适。将大问题分解为小问题，可以降低解决问题的难度，有助于逐步攻克各个小问题。另外，将复杂问题分解为若干小题，在调试和代码时可以单独调试，有利于最终整合。程序出现问题时也可以更快找到修改的目标代码，有利于代码的调试与维护。

5.3 汉诺塔完成了若干小题，总结你在完成过程中是否考虑了前后小题的关联关系，是否能尽可能做到后面小题有效利用前面小题已完成的代码，如何才能更好地重用代码？

在完成汉诺塔相关题目时，我首先考虑了前后小题本次大作业与之前汉诺塔题目之间的联系。比如此次大作业的前 4 个菜单项就是之前作业已经实现过的。因此只需将之前的代码根据新的限制条件进行修改即可使用。另外，后面的菜单项也有很多功能可以直接使用之前的代码。

为了有效利用前面小题已完成的代码，可以考虑将实现某个功能且使用频率高的部分代码封装成为一个函数，只需调用函数即可重复使用，不仅减少代码量，也更利于编写。同时为了能在更多地方使用已有函数，一个函数最好具有独立性，只实现一个特定的功能。

5.4 以本次作业为例，说明如何才能更好地利用函数来编写复杂的程序

首先，一个函数的封装要做到功能单一，确保每个函数只完成一个特定的功能，避免函数功能过于复杂。这样才能够增加函数的普适性，使函数的使用条件更少，使用频率更高。比如本次汉诺塔作业中，可以将汉诺塔递归、画柱子、盘子移动、横向数组打印、纵向数组打印、延时时间调整等代码封装成函数。

还有，可以为函数提供明确的参数，使得函数能够灵活应对不同的输入。比如调用重复度高的函数：递归函数、输入函数、输出函数等，提供 flag 参数代表不同的菜单项调用，然后在函数中通过参数调整功能，能有效增加函数的使用场合。比如本次作业中，输入函数通过参数判断是否要读取速度，使输入函数能同时在需要和不需要获取速度的模式函数中调用。还有输出函数，通过参数判断是否输出步数、横向数组、纵向数组、图形等，使其能在所有模式函数中调用。通过参数明确同一个函数的不同功能，能大大增加函数的适用性，也增加了代码的可读性。

6. 附件：源程序

```
void hanoi(int n, char src, char tmp, char dst,int flag)
{
    if (n == 1) {
        shuchu(n, src, tmp, dst, flag);
    }
    else {
        hanoi(n - 1, src, dst, tmp,flag);
        shuchu(n, src, tmp, dst, flag);
        hanoi(n - 1, tmp, src, dst,flag);
    }
}

void change_quanju(char src, char dst)
{
    num[dst - 65][sum[dst - 65]] = num[src - 65][sum[src - 65] - 1];
    sum[dst - 65]++;
    sum[src - 65]--;
}

void print_shuzu()
{
    cout << "A:";
    for (int i = 0; i < sum[0]; i++) {
        cout << right << setw(2) << setfill(' ') << num[0][i];
    }
    for (int i = sum[0]; i < 10; i++) {
        cout << " ";
    }
    cout << " ";
    cout << "B:";
    for (int i = 0; i < sum[1]; i++) {
        cout << right << setw(2) << setfill(' ') << num[1][i];
    }
    for (int i = sum[1]; i < 10; i++) {
        cout << " ";
    }
    cout << " ";
    cout << "C:";
    for (int i = 0; i < sum[2]; i++) {
        cout << right << setw(2) << setfill(' ') << num[2][i];
    }
    for (int i = sum[2]; i < 10; i++) {
        cout << " ";
    }
}

void shuchu(int n, char src, char tmp, char dst, int flag)
{
    cct_setcursor(CURSOR_INVISIBLE);
    if (flag == 1) {
        cout << right << setw(2) << setfill(' ') << n << "# " << src << "→" << dst << endl;
    }
    else if (flag == 2) {
```

```

        cout << "第" << right << setw(4) << setfill(' ') << (++step) << " 步(" << setw(2) << setfill(' ') << n << " #: " << src << "-->" << dst << ")" << endl;;
    }
    else if (flag == 3) {
        cout << "第" << right << setw(4) << setfill(' ') << (++step) << " 步(" << setw(2) << setfill(' ') << n << " #: " << src << "-->" << dst << ")" << endl;

        change_quanju(src, dst);
        print_shuzu();
        cout << endl;
    }
    else if (flag == 4) {
        stop(speed);

        cct_gotoxy(20, 20);
        cout << "第" << right << setw(4) << setfill(' ') << (++step) << " 步(" << setw(2) << setfill(' ') << n << " #: " << src << "-->" << dst << ")" << endl;

        change_quanju(src, dst);
        print_shuzu();
        stop(speed);
        cct_gotoxy(21+10*(src-'A'), 15 - sum[src - 'A']-1);
        cout << " ";
        cct_gotoxy(21 + 10 * (dst - 'A'), 15 - sum[dst - 'A']);
        cout << setw(2) << setfill(' ') << num[dst - 'A'][sum[dst - 'A']-1];
    }
    else if (flag == 8) {
        stop(speed);
        cct_gotoxy(20, 35);
        cout << "第" << right << setw(4) << setfill(' ') << (++step) << " 步(" << setw(2) << setfill(' ') << n << " #: " << src << "-->" << dst << ")" << endl;

        move_panzi(src, dst);
        change_quanju(src, dst);
        cct_gotoxy(41, 35);
        print_shuzu();
        cct_gotoxy(21 + 10 * (src - 'A'), 30 - sum[src - 'A'] - 1);
        cout << " ";
        cct_gotoxy(21 + 10 * (dst - 'A'), 30 - sum[dst - 'A']);
        cout << setw(2) << setfill(' ') << num[dst - 'A'][sum[dst - 'A'] - 1];
    }
    else if (flag == 9) {
        cct_gotoxy(20, 35);
        cout << "第" << right << setw(4) << setfill(' ') << (++step) << " 步(" << setw(2) << setfill(' ') << n << " #: " << src << "-->" << dst << ")" << endl;

        move_panzi(src, dst);
        change_quanju(src, dst);
        cct_gotoxy(41, 35);
        print_shuzu();
        cct_gotoxy(21 + 10 * (src - 'A'), 30 - sum[src - 'A'] - 1);
        cout << " ";
        cct_gotoxy(21 + 10 * (dst - 'A'), 30 - sum[dst - 'A']);
        cout << setw(2) << setfill(' ') << num[dst - 'A'][sum[dst - 'A'] - 1];
    }
    cct_setcursor(CURSOR_VISIBLE_NORMAL);
}

void move_panzi(char src, char dst)
{

```

```

    cct_setcursor(CURSOR_INVISIBLE);
    int n = src - 65;
    int m = dst - 65;
    for (int i = sum[n]; i < 13; i++) {
        cct_gotoxy(13 + 33 * (n) - num[n][sum[n] - 1], 15 - i);
        cct_setcolor(0, 0);
        for (int j = 0; j < num[n][sum[n] - 1]; j++) {
            cout << " ";
        }
        cct_setcolor(14, 14);
        cout << " ";
        cct_setcolor(0, 0);
        for (int j = 0; j < num[n][sum[n] - 1]; j++) {
            cout << " ";
        }
        stop(speed);
        cct_setcolor(num[n][sum[n]-1], num[n][sum[n] - 1]);
        cct_gotoxy(13 + 33 * (n) - num[n][sum[n] - 1], 15 - i-1);
        for (int j = 0; j < (num[n][sum[n] - 1]) * 2 + 1; j++) {
            cout << " ";
        }
        stop(speed);
    }

    if (dst > src) {
        for (int i = 0; i < 33*(dst-src); i++) {
            cct_gotoxy(13 + 33 * (n) - num[n][sum[n] - 1]+i, 2);
            cct_setcolor(0, 0);
            for (int j = 0; j < (num[n][sum[n] - 1])*2+1; j++) {
                cout << " ";
            }
            stop(speed);
            cct_setcolor(num[n][sum[n] - 1], num[n][sum[n] - 1]);
            cct_gotoxy(13 + 33 * (n)-num[n][sum[n] - 1] + i+1, 2);
            for (int j = 0; j < (num[n][sum[n] - 1]) * 2 + 1; j++) {
                cout << " ";
            }
            stop(speed);
        }
        cct_gotoxy(13 + 33 * (m) - (num[n][sum[n] - 1]), 2);
        cct_setcolor(0, 0);
        for (int j = 0; j < (num[n][sum[n] - 1]) * 2 + 1; j++) {
            cout << " ";
        }
        stop(speed);
        for (int i = 3; i < 15 - sum[m]; i++) {
            cct_gotoxy(13 + 33 * (m) - (num[n][sum[n] - 1]), i);
            cct_setcolor(num[n][sum[n] - 1], num[n][sum[n] - 1]);
            for (int j = 0; j < (num[n][sum[n] - 1]) * 2 + 1; j++) {
                cout << " ";
            }
            stop(speed);
            cct_gotoxy(13 + 33 * (m) - (num[n][sum[n] - 1]), i);
            cct_setcolor(0, 0);
            for (int j = 0; j < num[n][sum[n] - 1]; j++) {
                cout << " ";
            }
        }
    }

```

```

        cct_setcolor(14, 14);
        cout << " ";
        cct_setcolor(0, 0);
        for (int j = 0; j < num[n][sum[n] - 1]; j++) {
            cout << " ";
        }
        stop(speed);
    }
    cct_gotoxy(13 + 33 * (m) - (num[n][sum[n] - 1]), 15 - sum[m] - 1);
    cct_setcolor(num[n][sum[n] - 1], num[n][sum[n] - 1]);
    for (int j = 0; j < (num[n][sum[n] - 1]) * 2 + 1; j++) {
        cout << " ";
    }
}
else if (dst < src) {
    for (int i = 0; i < 33 * (src - dst); i++) {
        cct_gotoxy(13 + 33 * (n) - num[n][sum[n] - 1] - i, 2);
        cct_setcolor(0, 0);
        for (int j = 0; j < (num[n][sum[n] - 1]) * 2 + 1; j++) {
            cout << " ";
        }
        stop(speed);
        cct_setcolor(num[n][sum[n] - 1], num[n][sum[n] - 1]);
        cct_gotoxy(13 + 33 * (n) - num[n][sum[n] - 1] - i - 1, 2);
        for (int j = 0; j < (num[n][sum[n] - 1]) * 2 + 1; j++) {
            cout << " ";
        }
        stop(speed);
    }
    cct_gotoxy(13 + 33 * (m) - (num[n][sum[n] - 1]), 2);
    cct_setcolor(0, 0);
    for (int j = 0; j < (num[n][sum[n] - 1]) * 2 + 1; j++) {
        cout << " ";
    }
    stop(speed);
    for (int i = 3; i < 15 - sum[m]; i++) {
        cct_gotoxy(13 + 33 * (m) - (num[n][sum[n] - 1]), i);
        cct_setcolor(num[n][sum[n] - 1], num[n][sum[n] - 1]);
        for (int j = 0; j < (num[n][sum[n] - 1]) * 2 + 1; j++) {
            cout << " ";
        }
        stop(speed);
        cct_gotoxy(13 + 33 * (m) - (num[n][sum[n] - 1]), i);
        cct_setcolor(0, 0);
        for (int j = 0; j < num[n][sum[n] - 1]; j++) {
            cout << " ";
        }
        cct_setcolor(14, 14);
        cout << " ";
        cct_setcolor(0, 0);
        for (int j = 0; j < num[n][sum[n] - 1]; j++) {
            cout << " ";
        }
        stop(speed);
    }
    cct_gotoxy(13 + 33 * (m) - (num[n][sum[n] - 1]), 15 - sum[m] - 1);
    cct_setcolor(num[n][sum[n] - 1], num[n][sum[n] - 1]);

```

```

        for (int j = 0; j < (num[n][sum[n] - 1]) * 2 + 1; j++) {
            cout << " ";
        }
    }
    cct_setcolor(0, 7);
    cct_setcursor(CURSOR_VISIBLE_NORMAL);
}

```

```

void zhuzi()
{
    cct_setcursor(CURSOR_INVISIBLE);
    cct_setcolor(14, 14);
    cct_gotoxy(2, 15);
    for (int i = 0; i < 23; i++) {
        cout << " ";
    }
    cct_gotoxy(35, 15);
    for (int i = 0; i < 23; i++) {
        cout << " ";
    }
    cct_gotoxy(68, 15);
    for (int i = 0; i < 23; i++) {
        cout << " ";
    }
    for (int i = 1; i <= 12; i++) {
        for (int j = 0; j <= 2; j++) {
            cct_gotoxy(13+j*33, 15-i);
            cout << " ";
            Sleep(30);
        }
    }
    cct_setcolor(0, 7);
    cct_setcursor(CURSOR_VISIBLE_NORMAL);
}

```

```

void panzi(char a)
{
    cct_setcursor(CURSOR_INVISIBLE);
    int color = sum[a - 'A'];
    for (int i = 0; i < sum[a - 'A']; i++) {
        cct_gotoxy(13 + 33 * (a - 'A') - num[a - 'A'][i], 15 - i - 1);
        cct_setcolor(color, color);
        color -= 1;
        for (int j = 0; j < (num[a - 'A'][i]) * 2 + 1; j++) {
            cout << " ";
        }
        Sleep(50);
    }
    cct_setcolor(0, 7);
    cct_setcursor(CURSOR_VISIBLE_NORMAL);
}

```